

Consegna 8-ubuntu_hpc

L'obiettivo di questa esperienza laboratoriale è l'utilizzo di **Apptainer** all'interno del cluster di Ateneo. A differenza di Docker, che richiede l'esecuzione di un demone con privilegi di *root*, Apptainer è progettato specificamente per ambienti HPC multi-utente. Esso rispetta i privilegi dell'utente host, garantendo la sicurezza del nodo, e favorisce la riproducibilità della ricerca scientifica comprimendo l'intero ambiente in un singolo file eseguibile (`.sif`).

Modalità Interattiva e File System Dinamico

Nella prima fase del laboratorio è stato fatto l'accesso interattivo al container `ubuntu-hpc_latest.sif` messo a disposizione nella directory condivisa.

Dopo aver allocato un terminale su un nodo tramite `srun`, il modulo è stato caricato e la shell avviata:

```
[mariaandrea.zoccoli@wn90 ~]$ module load apptainer
[mariaandrea.zoccoli@wn90 ~]$ apptainer exec /hpc/home/roberto.alfieri/SHARE/HPC2526/
Apptainer/ubuntu-hpc_latest.sif bash
```

```
Apptainer> cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.3 LTS"
[...]
Apptainer> pwd
/hpc/home/mariaandrea.zoccoli/HPC2526/8-ubuntu_hpc
```

Modalità Batch e Analisi dell'Overhead

Per testare l'efficienza computazionale del container, è stato sottomesso un job batch per eseguire lo scaling OpenMP del codice `omp_heat` analizzato nelle consegne precedenti.

Lo script istruisce SLURM a riservare 4 core. Il comando `bash -c` passato ad Apptainer permette di navigare nella directory corretta e lanciare la simulazione senza interazione manuale.

```
OMP, 1, 4096, 4096, 500, 7.385, 7.027, wn99
OMP, 2, 4096, 4096, 500, 3.920, 3.617, wn99
OMP, 4, 4096, 4096, 500, 2.973, 2.640, wn99
OMP, 8, 4096, 4096, 500, 3.061, 2.701, wn99
OMP, 16, 4096, 4096, 500, 3.127, 2.762, wn99
OMP, 32, 4096, 4096, 500, 3.169, 2.783, wn99
```

I tempi di esecuzione diminuiscono linearmente fino a 4 thread (2.973 s), ma subiscono un peggioramento istanziando 8, 16 o 32 thread. Sebbene il nodo `wn99` disponga di numerosi core fisici, Apptainer eredita rigidamente i limiti imposti dallo scheduler (tramite *cgroups*). Avendo richiesto esplicitamente `--cpus-per-task=4`, l'istanza OpenMP subisce un forte overhead da context-switching oltre i 4 thread, dimostrando che la containerizzazione non bypassa i vincoli di allocazione hardware dell'host.

Porting

Come esperienza di porting dimostrativa, si è scelto di importare l'immagine ufficiale Docker di **TensorFlow per GPU**.

L'immagine è stata convertita in formato `.sif` tramite il comando: `apptainer pull tf_2.11_gpu.sif`
`docker://tensorflow/tensorflow:2.11.0-gpu`

test_gpu.py

```
import tensorflow as tf
print("--- Test Apptainer + TensorFlow GPU ---")
print("Versione TensorFlow:", tf.__version__)
gpus = tf.config.list_physical_devices('GPU')
print("Numero di GPU rilevate:", len(gpus))
if len(gpus) > 0:
    for gpu in gpus: print(" -", gpu)
```

Questo script interroga il backend di TensorFlow per verificare che i driver CUDA e l'hardware sottostante siano visibili dall'interno del container.

tf_gpu.slurm

```
#!/bin/bash
#SBATCH --output=%x.%j.out
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --partition=gpu
#SBATCH --qos=gpu
#SBATCH --gres=gpu:1
#SBATCH --time=0-00:10:00

module load apptainer
apptainer exec --nv tf_2.11_gpu.sif python3 test_gpu.py
```

Questo script richiede a SLURM un nodo dotato di scheda video. Il parametro cruciale è il flag `--nv` passato ad Apptainer, che esegue un bind dinamico delle librerie NVIDIA e dei driver CUDA presenti sul nodo host, proiettandoli all'interno del container per permetterne l'utilizzo.

```
--- Test Apptainer + TensorFlow GPU ---
Versione TensorFlow: 2.11.0
Numero di GPU rilevate: 1
Dettagli GPU:
- PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')
```

'output dimostra l'avvenuto e corretto interfacciamento. Il container isolato è in grado di sfruttare a pieno le risorse di accelerazione hardware del cluster. L'overhead computazionale di questo approccio è nullo, garantendo portabilità assoluta del software e alte performance.